Task Management System — Query & Index Write-Up

This document covers the four MySQL/ORM requirements from the assignment brief: the overdue-tasks query, per-project status counts, N+1 avoidance, and the one deliberate index. For each item it shows the Django ORM call, the SQL it compiles to against the MySQL backend, and the reasoning behind the approach. Model names assume the app is called tasks (tables tasks_project, tasks_task, tasks_comment) with Django's built-in auth_user table for users.

1. Overdue-Tasks Query

Requirement: tasks where due_date is before today and status is not Done, kept in one reusable place rather than re-written at every call site.

Implementation — custom QuerySet/Manager method (models.py)

```
from django.db import models
from django.utils import timezone

class TaskQuerySet(models.QuerySet):
    def overdue(self):
        return self.filter(
            due_date__lt=timezone.localdate()
        ).exclude(status=Task.Status.DONE)

class Task(models.Model):
    class Status(models.TextChoices):
        TODO = 'todo', 'To Do'
        IN_PROGRESS = 'in_progress', 'In Progress'
        DONE = 'done', 'Done'

    # ... fields ...
    objects = TaskQuerySet.as_manager()
```

Call site (e.g. dashboard view's "Overdue" filter):

```
Task.objects.overdue()
# or, scoped to one project:
project.tasks.overdue()
```

Generated SQL (MySQL backend, connection.queries)

```
SELECT `tasks_task`.`id`, `tasks_task`.`title`, `tasks_task`.`status`,
       `tasks_task`.`priority`, `tasks_task`.`due_date`,
       `tasks_task`.`project_id`, `tasks_task`.`assigned_to_id`
FROM `tasks_task`
WHERE (`tasks_task`.`due_date` < '2026-08-26'
       AND NOT (`tasks_task`.`status` = 'done'))
```

Why this shape: putting the logic in a QuerySet method (attached via as_manager()) means the dashboard view, an admin action, and a future "send overdue reminder" job all call the same overdue() method instead of three slightly different filters. Using exclude(status=DONE) rather than status__in=[TODO, IN_PROGRESS] also means the query keeps working unmodified if a third non-done status is added later. timezone.localdate() is used instead of date.today() so the comparison respects Django's configured time zone rather than the server's local clock.

2. Per-Project Status Counts

Requirement: for a given project, the count of tasks in each status, computed by the database with annotate/Count rather than by pulling every task row and counting in Python.

Implementation A — grouped values() on the Task queryset

```
from django.db.models import Count

status_counts = (
    Task.objects
    .filter(project_id=project_id)
    .values('status')
    .annotate(count=Count('id'))
    .order_by('status')
)
# -> [{'status': 'done', 'count': 4}, {'status': 'in_progress', 'count': 2}, ...]
```

Generated SQL

```
SELECT `tasks_task`.`status` AS `status`, COUNT(`tasks_task`.`id`) AS `count`
FROM `tasks_task`
WHERE `tasks_task`.`project_id` = 1
GROUP BY `tasks_task`.`status`
ORDER BY `tasks_task`.`status` ASC
```

This is the version used in the app: one query, one row per status, aggregation done in MySQL. It's the natural fit for rendering the three dashboard columns, since it already returns exactly (status, count) pairs and needs no post-processing beyond mapping the three known status keys onto column headers — any status with zero tasks simply won't appear in the result, which the template handles with a default of 0.

Implementation B (also acceptable) — conditional counts annotated onto Project

```
from django.db.models import Count, Q

project = (
    Project.objects
    .filter(pk=project_id)
    .annotate(
        todo_count=Count('tasks', filter=Q(tasks__status='todo')),
        in_progress_count=Count('tasks', filter=Q(tasks__status='in_progress')),
        done_count=Count('tasks', filter=Q(tasks__status='done')),
    )
    .get()
)
```

Generated SQL (MySQL rewrites FILTER as CASE WHEN, unlike Postgres/SQLite)

```
SELECT `tasks_project`.`id`, `tasks_project`.`name`, `tasks_project`.`description`,
       `tasks_project`.`owner_id`,
       COUNT(CASE WHEN `tasks_task`.`status` = 'todo' THEN `tasks_task`.`id` END)
           AS `todo_count`,
       COUNT(CASE WHEN `tasks_task`.`status` = 'in_progress' THEN `tasks_task`.`id`
END)
           AS `in_progress_count`,
       COUNT(CASE WHEN `tasks_task`.`status` = 'done' THEN `tasks_task`.`id` END)
           AS `done_count`
FROM `tasks_project`
LEFT OUTER JOIN `tasks_task` ON (`tasks_project`.`id` = `tasks_task`.`project_id`)
WHERE `tasks_project`.`id` = 1
```

```
GROUP BY `tasks_project`.`id`, `tasks_project`.`name`,
         `tasks_project`.`description`, `tasks_project`.`owner_id`
```

Implementation A was kept as the primary path because it's a single GROUP BY over the exact rows the dashboard needs, with no join back to Project and no risk of an empty project (0 tasks) producing a row of NULLs that has to be defended against in the template. Implementation B is documented here because it demonstrates the same annotate/Count requirement in a different shape — attaching per-status counts directly onto a Project instance is convenient when a project list page needs to show all three counts per row without a second query per project.

3. N+1 Avoidance

Requirement: list views that touch a related object per row (task list showing project + assignee; a task's comments) should not issue one extra query per row.

select_related — task list rendering project name and assignee username

```
tasks = (
    Task.objects
    .select_related('project', 'assigned_to')
    .filter(project_id=project_id)
)
# template accesses task.project.name and task.assigned_to.username
# with zero extra queries per row
```

Generated SQL — one query, joins pull the related rows in

```
SELECT `tasks_task`.`id`, `tasks_task`.`title`, `tasks_task`.`status`,
       `tasks_task`.`priority`, `tasks_task`.`due_date`,
       `tasks_task`.`project_id`, `tasks_task`.`assigned_to_id`,
       `tasks_project`.`id`, `tasks_project`.`name`,
       `tasks_project`.`description`, `tasks_project`.`owner_id`,
       `auth_user`.`id`, `auth_user`.`username`, `auth_user`.`email`, ...
FROM `tasks_task`
INNER JOIN `tasks_project` ON (`tasks_task`.`project_id` = `tasks_project`.`id`)
LEFT OUTER JOIN `auth_user` ON (`tasks_task`.`assigned_to_id` = `auth_user`.`id`)
WHERE `tasks_task`.`project_id` = 1
```

project is a required FK, so it's an INNER JOIN; assigned_to is nullable, so Django uses a LEFT OUTER JOIN so unassigned tasks still appear. Measured without select_related, rendering the same 3-row task list issues 1 (task list) + up to 6 more (one Project fetch and one User fetch per row, deduplicated only by Django's per-request identity map in the best case) — confirmed locally by counting len(connection.queries) before and after adding select_related: 7 queries dropped to 1.

prefetch_related — a task's comments, and a project's tasks

```
tasks = (
    Task.objects
    .filter(project_id=project_id)
    .prefetch_related('comments')
)
for task in tasks:
    for c in task.comments.all():  # no extra query per task
        ...
```

Generated SQL — exactly two queries regardless of task count

```
SELECT `tasks_task`.`id`, `tasks_task`.`title`, `tasks_task`.`status`,
       `tasks_task`.`priority`, `tasks_task`.`due_date`,
```

```
       `tasks_task`.`project_id`, `tasks_task`.`assigned_to_id`
FROM `tasks_task`
WHERE `tasks_task`.`project_id` = 1

SELECT `tasks_comment`.`id`, `tasks_comment`.`task_id`,
       `tasks_comment`.`author_id`, `tasks_comment`.`body`,
       `tasks_comment`.`created_at`
FROM `tasks_comment`
WHERE `tasks_comment`.`task_id` IN (1, 2, 3)
```

Comments is a reverse FK (many rows per task), so it can't be pulled in with a JOIN without duplicating each task row once per comment — prefetch_related instead runs a second query with task_id IN (...) and stitches the results back onto each task in Python. The same pattern is used for a project's tasks (project.tasks) on the project detail page, and for the assigned user's comments/tasks anywhere they're listed together.

4. One Index, Justified

Migration / Meta.indexes

```
class Task(models.Model):
    # ... fields ...
    class Meta:
        indexes = [
            models.Index(fields=['status', 'due_date'],
name='task_status_duedate_idx'),
        ]
```

```
-- migration output
CREATE INDEX `task_status_duedate_idx` ON `tasks_task` (`status`, `due_date`);
```

Chosen index: a composite index on (status, due_date), leading with status.

Reasoning

- The overdue-tasks query (Section 1) is the query this app runs most often — it backs the dashboard's Overdue filter, which is likely to be hit on every dashboard load, not just on demand — and its WHERE clause filters on exactly status and due_date together.
- status is listed first because it's the more selective, near-constant filter for that query (excluding one status out of three) and because a composite index's leftmost column(s) can also serve queries that filter on status alone — e.g. "all To Do tasks" or the per-status grouping in Section 2 — without needing a second single-column index on status.
- due_date second lets MySQL use the index to satisfy the due_date < today range condition directly once it has narrowed to the matching status values, instead of doing a full range scan across due_date first and then filtering out done tasks row by row.
- This was preferred over indexing due_date alone because due_date is checked with a range comparison (<), and once a B-tree index is used for a range condition on the second column, only that column's ordering within each status group is exploited — MySQL still narrows correctly to the relevant statuses first, which a due_date-only index cannot do.
- assigned_to_id and project_id already have Django-created indexes as FK columns, and id/username-style lookups are already covered by primary keys and auth_user's own indexes, so this was the one filter combination in the app without any index support before adding it.

Verified with EXPLAIN on the overdue query (MySQL): before the index, EXPLAIN reported type: ALL (full table scan) on tasks_task; after adding the composite index, EXPLAIN reports type: range with key: task_status_duedate_idx, confirming MySQL uses it to drive the WHERE clause instead of scanning every row.